



Universidad Zaragoza

Trabajo de Fin de Grado

Grado en Física

MACHINE LEARNING:

Métodos de ensamble aplicados a la estimación de la temperatura crítica en materiales superconductores

Marcos Barba Ballarín

Director: Ricardo López Ruiz

Zaragoza, 19 de mayo de 2026



*A José Luis Ballarín,
que espero se sintiera orgulloso.*

Agradecimientos

Este proyecto supone la culminación de una etapa que, con sus altibajos, ha conformado quién soy hoy más que ninguna otra. Recuerdo el día que atravesé por primera vez las puertas de la Universidad de Zaragoza y me enfrenté a mi primera sesión de clases como uno de los mayores golpes de realidad que he recibido: por primera vez, no había podido seguir al completo la explicación de un profesor, y esto se repitió durante la mayoría de las sesiones de aquella semana y las que vendrían posteriormente. La primera reacción que emergió de mi subconsciente fue la de abandonar, nunca había sido muy fuerte en ese aspecto, y en el pasado, si algo me había supuesto demasiado esfuerzo, simplemente había huido de allí.

Aquellos meses y en realidad, en varias ocasiones durante los dos primeros cursos, apareció esta idea en mi cabeza. Dado que hoy me encuentro escribiendo estas palabras, siento la necesidad de manifestar mi agradecimiento a varias personas, cuyo apoyo, soporte o simple presencia lo ha hecho posible.

A mis padres, que nunca dejaron de confiar en mí, y me mostraron su apoyo incondicional, fueran cuales fueran las circunstancias. En ocasiones pienso que ni siquiera escuchaban cuando intentaba argumentarles la difícil situación en la que me encontraba (cegado por el agobio de los exámenes) y sencillamente me respondían: “inténtalo, y si no, pues a la siguiente”.

A mi hermana, con la que en cierto modo he compartido este camino universitario. La infinidad de charlas sobre lo «jodidos que estábamos», en las que intentábamos quitarle importancia respectivamente a nuestras situaciones, me daban el empujón que necesitaba para pelear un poco más. Después de tanto luchar, las personas se hacen de acero, por eso estoy seguro de que conseguirás el éxito que persigues.

A mis abuelas, que creyeron siempre en su nieto, aunque no estaba de más encender una vela par pedir una pequeña ayuda del cielo.

A mis compañeros y amigos, porque con ellos he podido disfrutar de este camino. Sin ellos no habría tenido esos momentos tan necesarios para evadirme y recuperar fuerzas, mi fuente de energía estos años. Horas en la biblioteca, paradas por café, charletas en una habitación de la resi y alguna que otra noche mágica, son algunas de las experiencias que me llevo de estos años increíbles.

Resumen

El aprendizaje automático, ampliamente conocido por su acepción en inglés *machine learning*, vive hoy una época dorada, especialmente en el área del *deep learning* y las inteligencias artificiales generativas. No obstante, sin tanta repercusión mediática y antes de que estos modelos vieran la luz, las últimas décadas han estado lideradas por multitud de modelos predictivos que, sin alcanzar los increíbles resultados de las redes neuronales modernas, siguen siendo la elección preferida por muchos profesionales en ámbitos tan diversos como la investigación científica, la detección de fraude bancario, la predicción de mercados financieros o la toma de decisiones corporativas, entre otros.

En la mayoría de los casos, los modelos a los que actualmente nos referimos como *machine learning* «tradicional», no alcanzan los sorprendentes rendimientos de los modernos algoritmos basados en redes neuronales. Sin embargo, su estabilidad, su menor necesidad de datos e infraestructura computacional y, sobre todo, su explicabilidad, que nos permite interpretar cuál es la estrategia que el algoritmo ha encontrado para obtener las predicciones, los convierten en una opción realmente atractiva.

En el presente proyecto tomamos la iniciativa de estudiar algunos de los modelos más potentes de esta área del aprendizaje automático: aquellos algoritmos que, por medio de los conocidos como métodos de ensamble, combinan varios modelos en uno solo para lograr un mayor rendimiento. No sin antes revisar la estructura básica utilizada por todas estas técnicas: el árbol de decisión.

El objetivo será no solo acercarnos al funcionamiento de estas técnicas, sino también desarrollar una aplicación práctica, en este caso, en el ámbito de la investigación científica y, en particular, de la física de materiales: predecir la temperatura crítica en compuestos superconductores. Además, discutiremos las posibilidades que, tras el auge del *deep learning*, quedan para este tipo de modelos.

Índice general

Agradecimientos	v
Resumen	vii
1. Machine Learning	1
1.1. Tipos de aprendizaje	1
1.2. División del conjunto de datos	2
1.3. Hiperparámetros	2
1.4. Evaluación del modelo	2
2. Árboles de decisión	5
2.1. Entrenamiento del árbol	5
2.2. Árboles de regresión	6
2.3. Árboles de clasificación	7
2.4. Evitar el <i>overfitting</i>	8
2.4.1. <i>Early stopping</i>	8
2.4.2. <i>Tree pruning</i>	9
3. Métodos de ensamble	11
3.1. <i>Stacking</i>	11
3.2. <i>Bagging</i>	12
3.2.1. <i>Random Forest</i>	13
3.3. <i>Boosting</i>	14
3.3.1. <i>AdaBoost</i>	14
3.3.2. <i>Gradient Boosting</i>	15
3.3.3. <i>XGBoost</i>	16
4. Estimación de T_c	21
4.1. Introducción	21
4.2. Conjuntos de datos y descriptores	22
4.3. Modelado	23
4.3.1. Métricas de evaluación	23
4.3.2. Evaluación previa	24
4.4. Resultados	25

4.5. Conclusiones	27
A. Análisis exploratorio de los datos	31

Capítulo 1

Machine Learning

Aprender a partir de la experiencia, una habilidad que se atribuye generalmente a los seres vivos, especialmente a los pertenecientes al mundo animal y, en particular, a los seres humanos, es una de las características fundamentales que componen la inteligencia. Desde los inicios de la computación, se ha perseguido el objetivo de dotar a las máquinas de esta capacidad, en una constante búsqueda de alcanzar algún grado de **inteligencia artificial**.

El aprendizaje automático (*Machine Learning*) constituye un ámbito dentro de esta disciplina y hace referencia al conjunto de métodos y algoritmos que permiten que un sistema aprenda relaciones y patrones de forma autónoma, optimizando su funcionamiento con el objetivo de minimizar el error cometido en sus predicciones, a este proceso en el que el modelo «aprende» de la información que se le proporciona lo llamamos **entrenamiento**.

En función de los datos disponibles para el entrenamiento de un modelo, es posible distinguir dos tipos de aprendizaje: **supervisado** y **no supervisado**.

1.1. Tipos de aprendizaje

El **aprendizaje supervisado** es aquel en el que el conjunto de datos incluye ejemplos de entrada junto con sus correspondientes salidas o etiquetas. El objetivo es que el modelo aprenda las relaciones subyacentes entre ambas, de modo que, ante nuevas entradas para las cuales no se dispone de salida real, sea capaz de realizar una predicción adecuada. En función del tipo de variable que constituya la salida, los problemas pueden clasificarse como de **regresión**, cuando la salida es una variable numérica continua, o de **clasificación** cuando la salida es discreta y el modelo debe asignar cada muestra a una categoría o etiqueta determinada.

En contraste, el **aprendizaje no supervisado** se emplea cuando los datos no cuentan con etiquetas ni valores objetivo. Los modelos buscan descubrir estructuras o patrones subyacentes en los datos, agrupando observaciones similares o reduciendo la dimensionalidad del conjunto. Los algoritmos de *clustering* como *K-Means*, o los métodos de reducción de dimensionalidad como el *Análisis de Componentes Principales (PCA)* son ejemplos de aprendizaje no supervisado.

En el presente estudio vamos a centrarnos en técnicas de **aprendizaje supervisado**, dado

que constituye el enfoque más adecuado para problemas de predicción en los que disponemos de ejemplos con salida conocida. En concreto, profundizaremos en los modelos de **árboles de decisión** y en sus principales técnicas de ensamble.

1.2. División del conjunto de datos

En el aprendizaje supervisado es habitual dividir el conjunto de datos en dos partes: **entrenamiento** y **test**. El primero se utiliza durante el aprendizaje del modelo, es de este subconjunto del que el modelo extrae las relaciones y patrones subyacentes. El segundo se utiliza para evaluar la capacidad del modelo de generalizar a datos nunca antes vistos y determinar su capacidad predictiva.

Esta separación resulta esencial para evitar el **sobreajuste** (*overfitting*), es decir, que el modelo memorice los ejemplos de entrenamiento y pierda capacidad predictiva sobre información nueva.

Generalmente, entre un 70 % y un 80 % de los datos se destinan al entrenamiento, y el resto al test, aunque la proporción puede variar dependiendo del tamaño del conjunto de datos disponible.

1.3. Hiperparámetros

Como ya se ha mencionado, son los propios modelos los que ajustan de forma autónoma sus parámetros internos, adaptándose al conjunto de entrenamiento que se les proporciona. No obstante, los modelos de aprendizaje automático cuentan, en general, con una serie de parámetros externos denominados **hiperparámetros** que determinan su comportamiento y complejidad.

Estos parámetros deben fijarse previo al entrenamiento y su elección resulta crucial para el correcto funcionamiento del modelo, pues condicionará su capacidad de ajuste y generalización. Hiperparámetros mal seleccionados pueden derivar en modelos con exceso de **sesgo** o **varianza**. Usualmente, son seleccionados mediante pruebas sucesivas en las que se modifican sus valores hasta encontrar la combinación que proporciona el rendimiento óptimo del modelo.

1.4. Evaluación del modelo

Durante todo este capítulo se ha estado haciendo referencia a la evaluación del rendimiento del modelo, algo fundamental para cuantificar la capacidad de este de generalizar a nuevos datos. No obstante, todavía no habíamos profundizado en cuáles son las herramientas para llevar a cabo esta tarea.

La **validación** tradicional es una evaluación sobre el subconjunto de test, introducido en la sección 1.2. El modelo se ajusta en primer lugar utilizando el subconjunto de entrenamiento, y se evalúa su rendimiento con el subconjunto de test. De esta forma analizamos su funcionamiento con datos no vistos durante el entrenamiento. La principal limitación de este método es que la evaluación depende de una única división del conjunto de datos, lo que puede introducir cierta variabilidad.

Para mitigar este problema se recurre a **validación cruzada** (*cross-validation*) y, en particular, a su procedimiento más habitual: *k-fold cross-validation*. El conjunto de datos se divide en k particiones o *folds* de tamaño similar. El modelo se entrena k veces, utilizando en cada iteración $k - 1$ particiones para el entrenamiento y la restante para validación. Las métricas de evaluación se obtienen como la media de las obtenidas en cada validación. El resultado es una estimación más estable y representativa del rendimiento real del modelo.

Habitualmente, *cross-validation* se utiliza como una fase previa al entrenamiento final del modelo. Su propósito es obtener una estimación preliminar del rendimiento y optimizar hiperparámetros.

Capítulo 2

Árboles de decisión

En este capítulo se van a tratar los métodos de *machine learning* basados en árboles, tanto para problemas de regresión como de clasificación. La metodología se basa en la segmentación del espacio de predictores en regiones más simples y homogéneas. El objetivo es lograr estimaciones para cada región, basándonos en las variables respuesta de las muestras de entrenamiento que pertenecen a dicha región [1].

Todos estos métodos de partición del espacio pueden representarse mediante una estructura en forma de árbol. Las divisiones vienen dadas por nodos, donde se establecen condiciones basadas en los predictores. El resultado son una serie de nodos terminales u hojas, que representan cada una de las regiones segmentadas [1].

2.1. Entrenamiento del árbol

Ahora debemos tratar el proceso de construcción de un modelo de árbol. Como ya se dejó entrever en la introducción, nos encontramos con dos desafíos clave [1]:

- División del espacio de predictores, dado por el conjunto de vectores de observaciones $X_i, i = 1, \dots, l$, en K regiones independientes $R_1, R_2, \dots, R_K$.
- Determinar una predicción para cada región $R_k, k = 1, \dots, K$, basándonos en las variables respuesta de las observaciones de entrenamiento en R_k .

La estrategia seguida para que sea computacionalmente asumible entrenar árboles se denomina *Recursive binary splitting*, explicada por [2].

Recursive binary splitting

Sean el conjunto de entrenamiento $\mathcal{T} = \{(X_i, y_i)\}_{i=1}^l$, donde $X_i \in \mathbb{R}^p$ con p el número de características. Las variables respuesta son $y_i \in \mathbb{R}$ en caso de regresión y $y_i \in \mathcal{C}$ en caso de clasificación. Un árbol de decisión divide recursivamente el espacio de forma que las muestras con variables respuesta similares sean agrupadas conjuntamente.

Sea Q_m el conjunto de observaciones contenidas en el nodo m . Para cada posible división $\theta = (j, t_m)$, parametrizada por una de las p características j y un umbral t_m , se obtienen dos subconjuntos $Q_m^{left}(\theta)$ y $Q_m^{right}(\theta)$.

$$\begin{aligned} Q_m^{left}(\theta) &= \{(X_i, Y_i) | x_{i_j} \leq t_m\} \\ Q_m^{right}(\theta) &= \{(X_i, Y_i) | x_{i_j} > t_m\} \end{aligned}$$

Para un nodo m , *recursive binary splitting* considera los p predictores, y el posible valor umbral t_m para cada predictor. Una función de impureza $\mathcal{L}()$ es la encargada de evaluar cada posible división $\theta(j, t_m)$, eligiendo la que minimiza la impureza total ponderada:

$$\mathcal{I}(Q_m, \theta) = \frac{|Q_m^{left}|}{|Q_m|} \mathcal{L}(Q_m^{left}(\theta)) + \frac{|Q_m^{right}|}{|Q_m|} \mathcal{L}(Q_m^{right}(\theta)) \quad (2.1)$$

donde se tiene en cuenta el peso ponderado de cada subconjunto. La función de impureza $\mathcal{L}()$ viene determinada por las características del problema, principalmente si se trata de regresión o clasificación.

Esta metodología puede generalizarse al caso en que un predictor es categórico. Sea una variable categórica j , se elige un subconjunto S de categorías y el espacio de observaciones se divide según $x_{i_j} \in S$ o $x_{i_j} \notin S$.

Este algoritmo nos proporciona la división que minimiza localmente la impureza de cada división. No obstante, esta metodología no nos garantiza la obtención de la estructura óptima global para el árbol. A este tipo de estructuras se les denomina voraces (*greedy*). Los algoritmos voraces son, además, muy propensos al *overfitting*, en la sección 2.4 se tratarán estrategias para mitigar el problema.

2.2. Árboles de regresión

Cuando hablamos de árboles de regresión, nos referimos a aquellos que utilizamos para problemas de regresión, en los que la variable respuesta es continua.

A la hora de evaluar las posibles divisiones en un árbol de regresión, minimizamos una función de pérdida $\mathcal{L}()$. Según [3], los criterios a utilizar son el error cuadrático medio (MSE), la desviación de Poisson y el error medio absoluto (MAE). Para el nodo m , que contiene un conjunto de observaciones Q_m , podemos calcular la función de pérdida:

Error Cuadrático Medio (MSE):

$$\mathcal{L}(Q_m) = \frac{1}{|Q_m|} \sum_{y \in Q_m} (y - \bar{y}_m)^2$$

Desviación de Poisson:

$$\mathcal{L}(Q_m) = \frac{1}{|Q_m|} \sum_{y \in Q_m} (y \log \frac{y}{\bar{y}_m} - \bar{y}_m)$$

siendo necesario que $y \geq 0$. Utilizar la desviación de Poisson puede ser útil cuando se trata de conteos o frecuencias.

Tanto en el caso del error cuadrático medio (MSE) como en el de la Desviación de Poisson, la predicción asignada a un nodo terminal del árbol corresponde al valor medio de las variables respuesta y_i de las muestras contenidas en dicho nodo, ya que este valor minimiza la función de pérdida en ambos casos.

$$\bar{y}_m = \frac{1}{|Q_m|} \sum_{y \in Q_m} y$$

Error Medio Absoluto (MAE):

$$\mathcal{L}(Q_m) = \frac{1}{|Q_m|} \sum_{y \in Q_m} |y - \text{mediana}(y)_m|$$

Cuando usamos el error medio absoluto, para cada nodo terminal, se define como predicción la mediana $\text{mediana}(y)_m$ ¹ del subconjunto Q_m :

$$\text{mediana}(y)_m = \text{mediana}(y)_{y \in Q_m}$$

2.3. Árboles de clasificación

Utilizados para afrontar problemas de clasificación, estos árboles funcionan de forma muy similar a los de regresión. La diferencia más notable es que la variable respuesta ahora es categórica. En consecuencia, no solo se requiere un método distinto para obtener la predicción en cada región, sino que necesitaremos otros criterios para construirlos.

Basándonos en [3], existen distintos criterios que tienen el objetivo de encontrar los nodos más homogéneos/puros posibles. Sean $k = 1, 2, \dots, K$ las distintas clases que encontramos en el nodo m definimos p_{mk} :

$$p_{mk} = \frac{1}{|Q_m|} \sum_{y \in Q_m} 1\{y = k\}$$

como la proporción de observaciones que pertenecen a la clase k en el nodo m . Notar que $1\{y = k\}$ es una función indicadora, que toma el valor 1 cuando se cumple la condición, mientras que se anula para el resto de valores.

¹La mediana es el valor que divide un conjunto ordenado de datos en dos mitades con igual número de elementos.

Classification error rate:

Proporción de observaciones que no pertenecen a la clase mayoritaria del nodo.

$$\mathcal{L}(Q_m) = 1 - \max_k(p_{mk})$$

Sin embargo, no es suficientemente sensible para crear buenos árboles, por lo que no es usualmente empleado.

Índice de Gini: Cuantifica la impureza del conjunto en términos de la distribución de clases en este.

$$\mathcal{L}(Q_m) = Gini(Q_m) = \sum_{k=1}^K p_{mk}(1 - p_{mk})$$

Si $\mathcal{L}(Q_m) = 0$, el nodo m es completamente «puro», lo que significa que todos los ejemplos en ese nodo pertenecen a la misma clase. Si $\mathcal{L}(Q_m)$ se acerca a 1, el nodo es más «impuro», es decir, las muestras están más distribuidas entre las distintas clases.

Entropía: La entropía es una forma de cuantificar el desorden de un sistema.

$$\mathcal{L}(Q_m) = Entropy(Q_m) = - \sum_{k=1}^K \hat{p}_{mk} \log(\hat{p}_{mk})$$

En el caso de los nodos, el desorden se relaciona estrechamente con la impureza. Cuando un nodo contiene únicamente observaciones de una clase (puro), su entropía es cero. En cambio, cuando las clases están repartidas uniformemente en el nodo (totalmente impuro), su entropía es máxima (igual a 1).

En clasificación la predicción se basa en la clase mayoritaria dentro de la región, es decir, la moda de las observaciones en cada región. De igual forma, se suele acompañar del porcentaje de cada clase en dicha región, lo que nos aporta información sobre la confianza de la predicción.

$$\bar{y}_m = \arg \max_k p_{mk}$$

2.4. Evitar el *overfitting*

Los modelos descritos se ajustan muy bien a las observaciones de entrenamiento, reduciendo rápidamente su error asociado. Los árboles se ramifican adquiriendo estructuras complejas y generando sobreajuste (*overfitting*), perdiendo generalización. Como vemos planteado en [2], limitar el tamaño del árbol durante su construcción (*early stopping*) o añadir un proceso de podado (*pruning*) son las dos estrategias con las que contamos para prevenir este problema.

2.4.1. *Early stopping*

En [2], se introduce este método de regularización basado en la interrupción anticipada. Se establecen unas reglas de parada que limitan el tamaño final del árbol y que funcionan como

hiperparámetros del modelo. Las más comunes son:

- Observaciones mínimas para división.
- Observaciones mínimas de nodo terminal.
- Profundidad máxima del árbol: número de divisiones de la rama más larga.
- Número máximo de nodos terminales.
- Reducción mínima de error: establece una reducción mínima de error que tiene que lograr una división para realizarse.

2.4.2. *Tree pruning*

Como se explica en [4], la alternativa no *greedy* que consigue paliar el *overfitting* consiste en generar árboles grandes, minimizando las condiciones de parada, para después podarlos. El objetivo es mantener únicamente el subárbol óptimo, que minimiza el error de validación. Aplicar cross-validation a todas las posibles subestructuras de un árbol sería ideal, pero muy costoso computacionalmente. El algoritmo de *minimal cost-complexity pruning* realiza esta tarea de forma más eficiente.

Minimal cost-complexity pruning (MCCP):

Este método y las definiciones que se presentan a continuación se basan en el trabajo clásico de [2].

Sea T un árbol de decisión con un conjunto de nodos terminales $\tilde{T}$, definimos el coste del nodo t $R(t)$, y el coste total del árbol T , $R(T)$ como la suma de los costes de sus hojas:

$$R(T) = \sum_{t \in \tilde{T}} R(t)$$

Definimos también $\alpha > 0$ como el parámetro de complejidad, que penaliza el número de hojas del árbol; y la función de coste-complejidad:

$$R_\alpha(T) = R(T) + \alpha|\tilde{T}| \quad (2.2)$$

Sea T_t el subárbol de T cuya raíz es el nodo t . Podar el subárbol T_t significa reemplazarlo por un nodo terminal.

Entonces debemos preguntarnos cuándo resulta conveniente podar un subárbol. Para ello debemos comparar los costos con la ecuación 2.2:

- Mantener el subárbol T_t :

$$R_\alpha(T_t) = R(T_t) + \alpha|\tilde{T}_t|$$

- Podar el subárbol y dejar el nodo t como terminal:

$$R_\alpha(t) = R(t) + \alpha$$

Podemos podar T_t si

$$R_\alpha(t) \leq R_\alpha(T_t) \Rightarrow R(t) + \alpha \leq R(T_t) + \alpha|\tilde{T}_t|$$

y despejando α

$$\alpha \geq \frac{R(t) - R(T_t)}{|\tilde{T}_t| - 1} := \alpha_{eff}(t) \quad (2.3)$$

α_{eff} es la penalización límite, en la que ambos costes se equiparan.

El algoritmo de *minimal cost complexity pruning* calcula $\alpha_{eff}(t)$ para cada nodo t no terminal. El subárbol T_t , para cuya raíz t se cumple $\min(\alpha_{eff})$, es el elegido para la poda.

Este proceso se repite iterativamente, generando una secuencia de valores $\alpha_0 = 0 < \alpha_1 < \alpha_2 < \dots < \alpha_k$ que corresponden cada uno con un árbol podado $T_0 \supset T_1 \supset T_2 \supset \dots \supset T_k$, donde T_0 es el árbol completo y T_k el más podado.

Finalmente, utilizamos validación cruzada para seleccionar el valor óptimo de α . El α_i que mejor rendimiento demuestra es el parámetro óptimo y su árbol asociado T_i es el modelo elegido finalmente.

Cuando se utilizan modelos de aprendizaje automático, la atención suele centrarse en evitar el sobreajuste. No obstante, cuando el modelo, por falta de complejidad, es incapaz de capturar las relaciones subyacentes en nuestros datos, decimos que estamos ante un caso de *underfitting*. Este fenómeno, aunque menos frecuente, también debe considerarse. En el caso de los árboles de decisión, esto puede ocurrir si endurecemos mucho las condiciones de *early stopping*, dando lugar a modelos muy limitados en profundidad y anchura.

El objetivo es lograr un equilibrio entre complejidad y capacidad de generalización, ajustando los hiperparámetros para que el modelo sea estable, sin memorizar los datos de entrenamiento.

Capítulo 3

Métodos de ensemble

Basándonos en [5], las técnicas de *ensemble learning* se fundamentan en el acoplamiento de varios modelos simples con el objetivo de obtener un único modelo mucho más potente. Estos modelos más simples son ampliamente conocidos como modelos débiles o modelos base.

Como se indica en [5], un modelo débil es aquel cuya tasa de acierto supera ligeramente el azar. De esta forma, el *ensemble* sí puede lograr un mejor rendimiento con el aumento del número de modelos base en él.

Existen multitud de técnicas de *ensemble*, algunas más simples como el *max voting* para problemas de clasificación, o *averaging* para problemas de regresión. No obstante, en este estudio vamos a centrarnos en métodos más complejos como *stacking*, *bagging* y *boosting*, poniendo especial atención en estos dos últimos.

3.1. Stacking

Las predicciones de múltiples estimadores llamados modelos base, generalmente de distinta naturaleza, sirven finalmente como entrada de un estimador final, el *meta-model*, que computa la predicción final basándose en las salidas de los modelos base.

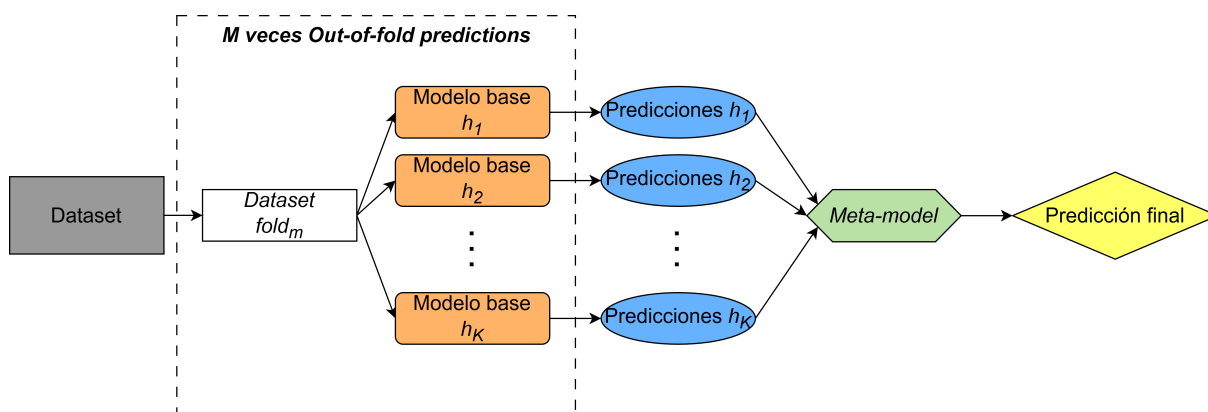


Figura 3.1: Diagrama del funcionamiento del *stacking*

Todo el desarrollo de esta técnica de ensamble se ha extraído de [6]. Sea $\mathcal{T} = \{(X_i, y_i)\}_{i=1}^n$ el conjunto de entrenamiento, donde $X_i \in \mathbb{R}^p$ es un vector de características y $y_i \in \mathbb{R}$ (regresión) o $y_i \in \mathcal{C}$ (clasificación) las variables respuesta. Sean $h_1, \dots, h_K$ los modelos base que se van a utilizar en el ensamble.

Entonces, cada estimador h_k se entrena de forma independiente sobre $\mathcal{T}$ y genera predicciones para cada X_i : $Z_{ik} = h_k(X_i)$. El resultado es un conjunto de datos $\mathcal{Z} = \{(Z_i, y_i)\}_{i=1}^n$, donde $Z_i = (h_1(X_i), \dots, h_K(X_i))$ funcionará como un conjunto de predictores para el *meta-model*. Finalmente, se entrena un *meta-model* H sobre $\mathcal{Z}$.

Ahora el modelo está preparado para generar una predicción y sobre una nueva muestra X :

$$y = H(h_1(X), \dots, h_K(X))$$

Los modelos base realizan las predicciones del mismo conjunto para el que son entrenados. La idea original de [6] fue utilizar *out-of-fold predictions*, para evitar entrenar el *meta-model* con predicciones sobreajustadas.

Se divide el conjunto $\mathcal{T}$ en M particiones. Para cada partición $fold_m$, se entrenan los K modelos base $h_k^{(-m)}$ sobre $\mathcal{T} \setminus fold_m$ y se predice sobre $fold_m$ ($h_k^{(-m)}$ indica que el modelo se entrenó sin usar el $fold_m$). Tras completar el proceso, se tienen predicciones no sesgadas de los K *base model* sobre las n muestras. Son estas predicciones las utilizadas para construir $\mathcal{Z}$ y entrenar el *meta-model*. Finalmente, los K modelos base h_k son entrenados sobre todo el conjunto de entrenamiento $\mathcal{T}$, y el modelo está preparado para obtener predicciones sobre nuevas muestras.

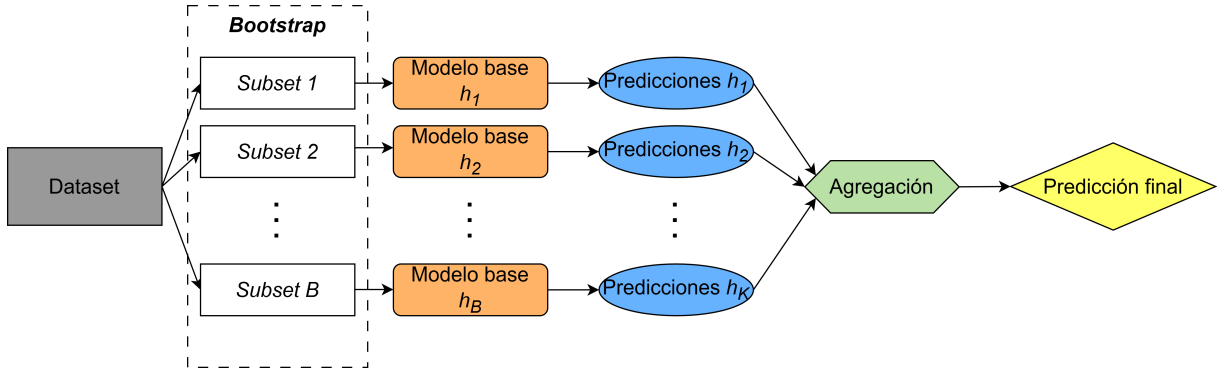
3.2. Bagging

El *Bagging* (abreviatura de *Bootstrap Aggregating*) es un ensamble homogéneo, con esto queremos decir que los modelos base tienen una estructura común. El objetivo es reducir la varianza a través de la combinación de múltiples modelos base, que son entrenados sobre distintas muestras generadas mediante remuestreo con reemplazo (*bootstrap*) del conjunto de entrenamiento original. La predicción final se obtiene agregando (*aggregating*) las predicciones individuales de los modelos base.

Desarrollando como en [7]. Sea $\mathcal{T} = \{(X_i, y_i)\}_{i=1}^n$ el conjunto de entrenamiento, donde $X_i \in \mathbb{R}^p$ es un vector de características e $y_i \in \mathbb{R}$ (regresión) o $y_i \in \mathcal{C}$ (clasificación) las variables respuesta. A partir de $\mathcal{T}$ y mediante *bootstrap* (remuestreo con reemplazo), se generan B subconjuntos $\mathcal{T}^{(1)}, \dots, \mathcal{T}^{(B)}$.

Se entrena un modelo base $h^{(b)}$ sobre cada subconjunto $\mathcal{T}^{(b)}$. La predicción final sobre una nueva muestra X se obtiene por agregación de las predicciones individuales de cada modelo base: promedio para regresión (izquierda) y clase más frecuente para clasificación (derecha).

$$H(X) = \frac{1}{B} \sum_{b=1}^B h^{(b)}(X) \qquad H(X) = \text{mode} \left\{ h^{(1)}(X), \dots, h^{(B)}(X) \right\}$$

Figura 3.2: Diagrama del funcionamiento del *Bagging*

Como se detalla en [1], existe una forma muy directa de estimar el error de un modelo de *bagging* sin tener que recurrir a *k-fold cross-validation*. Cada modelo base $h^{(b)}$ es entrenado sobre un subconjunto generado por *bootstrap* $\mathcal{T}^{(b)}$ del conjunto de entrenamiento original $\mathcal{T}$. En promedio, $\mathcal{T}^{(b)}$ contiene alrededor de $2/3$ de las observaciones en $\mathcal{T}$. Entonces, al conjunto $\mathcal{T} \setminus \mathcal{T}^{(b)}$ no utilizado durante el entrenamiento de $h^{(b)}$, que supone en torno a $1/3$ del conjunto de entrenamiento original, contiene las denominadas observaciones *out-of-bag* (OOB).

Para obtener la predicción para una observación (X_i, y_i) basta con utilizar aquellos modelos base $h^{(b)}$ cuyos subconjuntos de entrenamiento $\mathcal{T}^{(b)}$ no contengan dicha observación (X_i, y_i) es decir, los modelos base para los que dicha observación es OOB. La predicción OOB se obtiene por agregación de las predicciones de dichos modelos base para la i -ésima observación. Este proceso se repite para las n observaciones y las predicciones OOB son utilizadas para computar el error.

3.2.1. *Random Forest*

Como se detalla en [1] *Random Forest* es un algoritmo considerado una extensión del *bagging*. Conserva las características de este método de ensamble, utilizando como estimadores árboles de decisión, entrenándose cada uno con un subconjunto generado por *bootstrap* del conjunto de entrenamiento original. Además, obtiene una estimación final agregando, como en cualquier otro *bagging*, las predicciones de cada estimador.

La característica novedosa de este algoritmo es la implementación de un mecanismo explícito para reducir la correlación entre los múltiples árboles de decisión que componen el «bosque». Y es que para la división de cada nodo, solo se tendrá en cuenta un subconjunto aleatorio del total de características. Esta introducción de aleatoriedad adicional induce a los árboles a explorar nuevas estructuras de decisión. Se permite que los árboles adquieran profundidad y no se aplican procesos de poda, por lo que, de forma individual, cada árbol, presentará un sesgo bajo pero una varianza alta. Tras agregar todos los árboles, el modelo final presenta una menor varianza, generalizando mejor a nuevos datos.

Como lo encontramos en [8]. Sea $\mathcal{P} = \{1, \dots, p\}$ el conjunto de índices de las características. Recordando lo tratado en el apartado 2.1, una posible división venía dada por $\theta = (j, t_m)$, donde $j \in \mathcal{P}$, y t_m es el valor umbral establecido para la división. Durante el proceso de *recursive binary*

splitting tratado en 2.1, *Random Forest* introduce que para cada nodo se toma un subconjunto $\mathcal{F} \subset \mathcal{P}$ con $|\mathcal{J}| = k$ para evaluar la mejor división. De esta forma, cada posible división $\theta = (j, t_m)$ la característica se toma de este subconjunto $j \in \mathcal{F}$ y no del total de características.

3.3. Boosting

Demostrada su viabilidad teóricamente en [5], *boosting* es un método de ensamble que suele implementarse de forma homogénea. La diferencia sustancial con el *bagging* es que los modelos base no son independientes, sino que se construyen de forma secuencial y aditiva: cada nuevo modelo se entrena prestando mayor atención a las observaciones que han sido mal predichas por sus predecesores. El modelo final es una combinación ponderada de los modelos débiles, en la que los mejores modelos tienen más influencia. Este ensamble final logra, en general, reducir el sesgo.

La atención a las observaciones que presentan mayor error en las predicciones de los modelos anteriores se logra mediante la asignación de pesos, que se ajustan iterativamente para favorecer la corrección de los casos más difíciles.

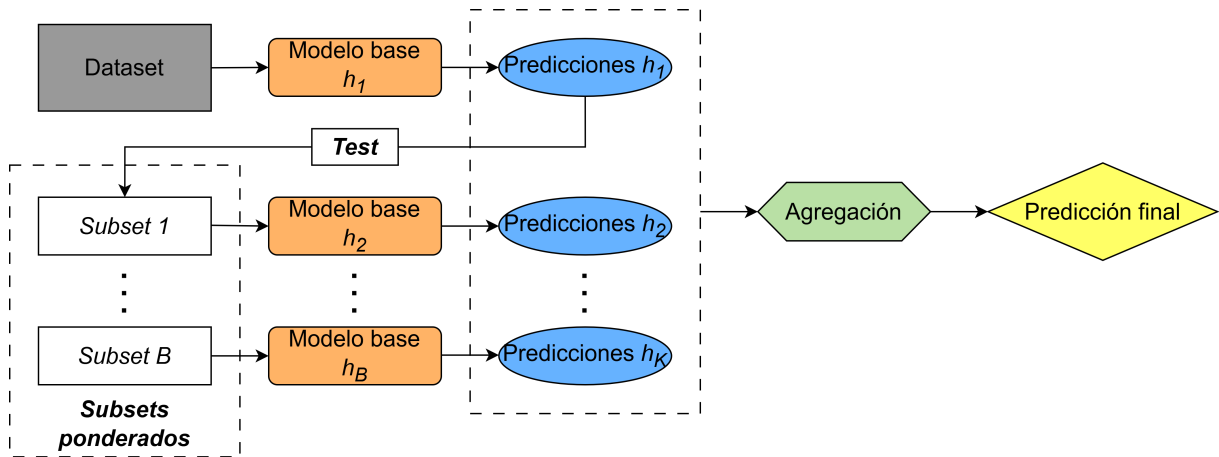


Figura 3.3: Diagrama del funcionamiento del *Boosting*

Existen múltiples algoritmos basados en esta metodología, como *AdaBoost*, *Gradient Boosting* o implementaciones modernas, como *XGBoost*, que difieren en la forma en que calculan el error de los modelos base, actualizan los pesos de las muestras o combinan finalmente los modelos.

3.3.1. AdaBoost

Introducido en [9], se trata del algoritmo pionero en aplicar esta estrategia de ensamble. Originalmente concebido para clasificación binaria, se utilizan, en general, *stumps* como modelos débiles, es decir, árboles de decisión de un solo nivel para conformar un clasificador binario fuerte.

Sea $\mathcal{T} = \{(X_i, y_i)\}_{i=1}^n$ el conjunto de entrenamiento, donde $X_i \in \mathbb{R}^p$ es un vector de características e $y_i \in \{-1, +1\}$ la variable de salida. Inicialmente, se asignan a las muestras $\{(X_i, y_i)\}_{i=1}^n$

pesos iguales:

$$w_i^{(1)} = \frac{1}{n} \quad \forall i = 1, \dots, n$$

Para cada iteración $t = 1, \dots, T$:

- Se entrena un clasificador débil $h_t(X)$ utilizando los pesos correspondientes $\{w_i^{(t)}\}_{i=1}^n$. En particular, cuando se utilizan *stumps* como modelos base, los pesos alteran el proceso de *recursive binary splitting*, ponderando sobre estos pesos y priorizando la correcta clasificación de las muestras con mayor peso.
- Se calcula el error ponderado:¹

$$\varepsilon_t = \frac{\sum_{i=1}^n w_i^{(t)} 1\{h_t(x_i) \neq y_i\}}{\sum_{i=1}^n w_i^{(t)}}$$

- Se asigna un peso al clasificador:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right)$$

que refleja su capacidad predictiva, cuanto menor es el error ε_t , mayor es el peso y, mayor será su influencia en el clasificador final.

- Se actualizan los pesos de las muestras:²

$$w_i^{(t+1)} = w_i^{(t)} \exp(-\alpha_t y_i h_t(X_i))$$

de forma que $w_i^{(t+1)} = w_i^{(t)} \exp(-\alpha_t)$ para muestras bien clasificadas, mientras que $w_i^{(t+1)} = w_i^{(t)} \exp(\alpha_t)$ cuando una muestra se clasifica erróneamente. Finalmente, se normalizan de forma que $\sum_{i=1}^n w_i^{(t+1)} = 1$.

El clasificador final $H(X)$ se obtiene como combinación lineal de los clasificadores débiles ponderados por sus pesos:

$$H(X) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(X) \right)$$

De esta forma, el algoritmo *AdaBoost* consigue, en general, reducir el sesgo iterativamente, al enfocarse progresivamente en las muestras más difíciles de clasificar en cada iteración.

Más adelante se introdujeron extensiones del algoritmo tanto para clasificación multiclase, como para regresión.

3.3.2. Gradient Boosting

Formalmente descrito en [10], *Gradient Boosting* es un método de ensamble basado en el mismo principio que *AdaBoost*. No obstante, a diferencia de este último, que actualiza los pesos de las

¹ $\sum_{i=1}^n w_i^{(t)} = 1 \forall t$ pues los pesos se normalizan tras cada actualización.

²Notar que los valores de $y_i, h_t(X_i) \in \{\pm 1\}$.

muestras en cada iteración, la estrategia de *Gradient Boosting* es plantear el proceso como un descenso de gradiente sobre una función de pérdida.

Sea $\mathcal{T} = \{(X_i, y_i)\}_{i=1}^n$ el conjunto de entrenamiento, donde $X_i \in \mathbb{R}^p$ es un vector de características e $y_i \in \mathbb{R}$ (regresión) ó $y_i \in \mathcal{C}$ (clasificación) las variables respuesta. Inicialmente, se define un modelo constante:

$$H_0(X) = \arg \min_c \sum_{i=1}^n \mathcal{L}(y_i, c)$$

En cada iteración $t = 1, \dots, T$:

- Se calcula el gradiente negativo de la función de pérdida para cada muestra, lo que denominamos **pseudoresiduos**:

$$r_{it} = - \left[\frac{\partial \mathcal{L}(y, H(X_i))}{\partial H(X_i)} \right]_{H(X_i)=H_{t-1}(X_i)} \quad i = 1, \dots, n$$

notar que como $H(X_i)$ utilizamos el modelo ensamblado hasta la iteración anterior $t - 1$.

- Se entrena un nuevo modelo débil $h_t(X)$, generalmente un árbol de decisión poco profundo, utilizando como variables objetivo los pseudoresiduos calculados previamente. Es decir, en cada iteración utilizamos el conjunto de entrenamiento $\mathcal{T}^{(t)} = (X_i, r_{it})_{i=1}^n$.
- Se calcula el coeficiente γ_t óptimo que minimiza la función de pérdida. Este coeficiente determina la escala en que contribuye la dirección de descenso de gradiente estimada por h_t .

$$\gamma_t = \arg \min_{\gamma} \sum_i \mathcal{L}(y_i, H_{t-1}(X_i) + \gamma h_t(X_i))$$

- Se actualiza el modelo ensamblado con una tasa de aprendizaje $\nu \in (0, 1]$, que es un hiperparámetro fijo que se escoge para todo el proceso de ensemble.

$$H_t(X) = H_{t-1}(X) + \nu \gamma_t h_t(X)$$

.

Este proceso se repite hasta que se alcanza un criterio de parada, normalmente que la reducción de la pérdida sea mínima, o que el error en validación deje de reducirse.

3.3.3. *XGBoost*

Extreme Gradient Boosting, ampliamente conocido por su acrónimo *XGBoost*, es un algoritmo de código abierto planteado como una extensión de *Gradient Boosting*, en la que se implementan mejoras de rendimiento y optimización. Fue introducido formalmente en [11], donde se detallan los principales aportes de este algoritmo. El proyecto mantiene una implementación activa que

podemos consultar en *GitHub*³.

Aunque a nivel práctico amplía el concepto de *Gradient Boosting*, este algoritmo plantea un cambio estructural que afecta a la forma en que se construyen los árboles, por ello es conveniente introducir el concepto de *Extreme Gradient Boosted Tree (XGBTree)*.

XGBTree

Por cuestión de claridad, realizamos un cambio de notación y el árbol de decisión entrenado en la iteración t vendrá dado por f_t . Además, su predicción para la muestra i vendrá dada por $\hat{y}_i^{(t)}$:

$$\begin{aligned}\hat{y}_i^{(1)} &= f_1(X_i) \\ \hat{y}_i^{(2)} &= f_1(X_i) + f_2(X_i) = \hat{y}_i^{(1)} + f_2(X_i) \\ &\vdots \\ \hat{y}_i^{(t)} &= \sum_t f_t(X_i) = \hat{y}_i^{(t-1)} + f_t(X_i)\end{aligned}$$

El cambio fundamental a la hora de entrenar estos árboles es que a la hora de construirse, no nos limitamos a elegir como variable objetivo unos pseudoresiduos, sino que se introduce el descenso del gradiente en la misma función de pérdida que usamos para evaluar cada división del árbol. Además, se añade un término de regularización $\Omega(f_t)$ que controla la complejidad y el número de hojas del árbol. Para una iteración t , evaluamos la que vamos a denominar **función objetivo** $\mathcal{L}^{(t)}$, que tendrá que minimizarse durante el entrenamiento del árbol:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \ell(y_i, \hat{y}_i^{(t-1)} + f_t(X_i)) + \Omega(f_t)$$

donde ℓ es la función de pérdida empírica (error cuadrático, log-loss, ...).

Ahora, podemos tomar la expansión de Taylor hasta segundo orden de la función de pérdida:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \left[\ell(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(X_i) + \frac{1}{2} h_i f_t(X_i)^2 \right] + \Omega(f_t)$$

donde como primera y segunda derivada de la función de pérdida están relacionadas con gradiente y hessiano, se utiliza la notación g_i y h_i , que se definen:

$$g_i =: \frac{\partial \ell(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}} \quad h_i =: \frac{\partial^2 \ell(y_i, \hat{y}_i^{(t-1)})}{\partial^2 \hat{y}_i^{(t-1)}}$$

Tras eliminar las constantes, la función objetivo resulta:

³<https://github.com/dmlc/xgboost>

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \left[g_i f_t(X_i) + \frac{1}{2} h_i f_t(X_i)^2 \right] + \Omega(f_t)$$

donde g_i y h_i dependen de la predicción anterior, mientras que $f_t(X_i)$ hace referencia a la predicción del árbol que estamos construyendo. Podemos interpretar este entrenamiento como un árbol que no solo sigue la dirección del gradiente, sino que también tiene en cuenta la curvatura de la función de pérdida, con lo que se consigue mejor precisión.

La función objetivo cuenta además con un término de regularización $\Omega(h_t)$. En primer lugar, redefinimos el árbol f_t :

$$f_t(X_i) = w_{q(X_i)}, w \in R^J, q : R^n \rightarrow R^J$$

donde w es un vector con las predicciones correspondientes a cada hoja del árbol f_t y q una función que asigna a cada muestra X_i una hoja del árbol, con J el número total de hojas. Definimos el término de regularización como:

$$\Omega(f_t) = \gamma J + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2$$

donde el término γJ penaliza el número de hojas, favoreciendo árboles más sencillos y evitando divisiones que no tengan aportes significativos en la reducción de pérdida, mientras que el término $\frac{1}{2} \lambda \sum_{j=1}^J w_j^2$ limita la magnitud de la predicción de las hojas, impidiendo pasos demasiado grandes en la dirección del gradiente.

Con esta reformulación, podemos reescribir la función objetivo para el t –ésimo árbol como:

$$\begin{aligned} \mathcal{L}^{(t)} &= \sum_{i=1}^n \left[g_i w_{q(X_i)} + \frac{1}{2} h_i w_{q(X_i)}^2 \right] + \gamma J + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2 \\ &= \sum_{j=1}^J \left[\left(\sum_{i \in Q_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in Q_j} h_i + \lambda \right) w_j^2 \right] + \gamma J \end{aligned}$$

donde Q_j es el conjunto de muestras en la hoja j , es decir, $i \in Q_j$ son las muestras en la hoja j . Notar entonces que hemos cambiado el índice del sumatorio, pues todas las muestras de la hoja j obtienen la misma predicción. Podemos ahora compactar la expresión definiendo $G_j = \sum_{i \in Q_j} g_i$ y $H_j = \sum_{i \in Q_j} h_i$:

$$\mathcal{L}^{(t)} = \sum_{j=1}^J \left[G_j w_j + \frac{1}{2} H_j w_j^2 \right] + \gamma J$$

Como las w_j son independientes, para una estructura de árbol $q(X_i)$ dada, podemos obtener analíticamente la reducción óptima de la función objetivo:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

$$\mathcal{L}^{(t)*} = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma J$$

$\mathcal{L}^{(t)*}$ nos sirve como una suerte de puntuación de que tan buena es una estructura de árbol $q(X_i)$.

Con esto ya tenemos un marco para medir la calidad de un árbol. Idealmente, generaríamos todos los posibles árboles y nos quedaríamos con el mejor. No obstante, en la práctica esto es inviable, por lo que recurrimos a estrategias de optimización para cada división. Definimos la ganancia de una división como:

$$Gain = \frac{1}{2} \left[\underbrace{\frac{G_L^2}{H_L + \lambda}}_{(1)} + \underbrace{\frac{G_R^2}{H_R + \lambda}}_{(2)} - \underbrace{\frac{(G_L + G_R)^2}{H_L + H_R + \lambda}}_{(3)} \right] - \underbrace{\gamma}_{(4)}$$

donde el término (1) es la puntuación de la hoja izquierda, (2) es la puntuación de la hoja derecha y (3) es la puntuación de la hoja original. (4) es el término de regularización correspondiente a la adición de una nueva hoja: si la ganancia es menor que γ , no se añadirá esa rama (se comporta como un término de poda).

Sea $\mathcal{T} = \{(X_i, y_i)\}_{i=1}^n$ el conjunto de entrenamiento, donde $X_i \in \mathbb{R}^p$ es un vector de características e $y_i \in \mathbb{R}$ (regresión) ó $y_i \in \mathcal{C}$ (clasificación) las variables respuesta. Inicialmente, se define un modelo constante:

$$F_0(X) = \arg \min_c \sum_{i=1}^n \mathcal{L}(y_i, c)$$

En cada iteración $t = 1, \dots, T$:

- Se computan los gradientes g_i y hessianas h_i correspondientes a todas las muestras.⁴

$$g_i =: \frac{\partial \ell(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}} \quad h_i =: \frac{\partial^2 \ell(y_i, \hat{y}_i^{(t-1)})}{\partial^2 \hat{y}_i^{(t-1)}}$$

- Se entena el árbol f_t utilizando los g_i y h_i para calcular la ganancia de cada división.
- Se actualiza el modelo ensamblado utilizando la tasa de aprendizaje $\eta \in (0, 1]$:

$$F_t(X) = F_{t-1}(X) + \eta f_t$$

Finalmente, el modelo ensamblado final es $F_T(X) = F_0(X) + \eta \sum_{t=1}^T f_t$.

⁴Recordemos que $\hat{y}_i^{(t-1)}$ corresponde con la predicción $f_{t-1}(X_i)$.

Capítulo 4

Estimación de la temperatura crítica de superconductores

4.1. Introducción

La resistividad eléctrica de un conductor metálico disminuye gradualmente cuando la temperatura se reduce. En conductores ordinarios, se alcanza un valor límite, de modo que mantienen una resistencia no nula incluso cerca del cero absoluto. La resistencia de un superconductor, en cambio, desciende bruscamente a cero cuando el material se enfría por debajo de su temperatura crítica (T_c).

En 1911, el físico neerlandés Heike Kamerlingh Onnes observó que la resistencia eléctrica del mercurio desaparecía bruscamente al enfriarse a 4K. Este hallazgo, junto con el desarrollo de las técnicas necesarias para licuar helio y alcanzar dichas temperaturas, le valieron el premio Nobel de Física. Al tratarse la superconductividad de un fenómeno esencialmente cuántico, tuvieron que pasar varias décadas hasta que se produjeron los primeros avances en la comprensión de este fenómeno: las teorías BCS y de Ginzburg-Landau.

Tras algunos años sin avances significativos, se descubrió una familia de materiales cerámicos, los óxidos de cobre con estructura de perovskita, que eran superconductores con temperaturas críticas superiores a 90K. Estos materiales, conocidos como superconductores de alta temperatura, abrieron la puerta a aplicaciones tecnológicas viables, al mantener la superconductividad en condiciones más accesibles.

Entre sus aplicaciones más destacadas se encuentran los equipos de resonancia magnética nuclear utilizados en hospitales, los sistemas de transporte por levitación magnética y, en el ámbito científico, el direccionamiento de haces en aceleradores de partículas o la fabricación de qubits para computadores cuánticos.

No obstante, la predicción teórica de la temperatura crítica de un superconductor continúa siendo un problema abierto más de un siglo después de su descubrimiento. Ante la ausencia de modelos predictivos basados en la teoría, la experimentación empírica ha guiado históricamente la síntesis de nuevos materiales superconductores. En este contexto, el aprendizaje automático

(Machine Learning) se perfila como una herramienta prometedora para abordar este desafío. En este estudio, utilizaremos algunas de las técnicas explicadas con anterioridad con el objetivo de estimar la temperatura crítica (T_c) de superconductores a partir de sus propiedades físicas y químicas, extraídas directamente de su fórmula química.

Los datos empleados provienen originalmente de la base de datos de materiales superconductores mantenida por el Instituto Nacional para la Ciencia de Materiales (NIMS) de Japón. En particular, el conjunto de datos utilizado en este trabajo es el resultado de una cuidadosa minería y depuración de datos realizada sobre dicha base original, descrita detalladamente en [12], de la que se han extraído una serie de descriptores físico-químicos interpretables por los algoritmos de aprendizaje automático.

4.2. Conjuntos de datos y descriptores

El conjunto de datos que vamos a utilizar cuenta con 81 características extraídas de 21.263 superconductores cuya temperatura crítica es conocida. Inicialmente, se tienen 8 propiedades referidas a los elementos que conforman los compuestos:

- **Masa atómica** [UMA]: masa atómica del elemento en reposo.
- **Primera energía de ionización** (*fie*) [kJ/mol]: energía requerida para quitar de un átomo un electrón de valencia
- **Radio atómico** [pm]: radio del átomo del elemento.
- **Densidad** [kg/m^3]: densidad del elemento en condiciones estándar de presión y temperatura.
- **Afinidad electrónica** [kJ/mol]: energía requerida para añadir un electrón a un átomo neutro.
- **Calor de fusión** [kJ/mol]: energía necesaria para cambiar de fase sólida a líquida el elemento sin cambio de temperatura (calor latente de fusión).
- **Conductividad térmica** [$W/(m \cdot K)$]: coeficiente (κ) de conductividad térmica del elemento.
- **Valencia**: número típico de enlaces que forma el elemento.

Con estas propiedades elementales se generan 10 familias de descriptores estadísticos referentes a los compuestos:

- | | |
|---|--|
| ▪ $\text{mean_}^* = \mu = \frac{1}{n} \sum_{i=1}^n t_i$ | ▪ $\text{wtd_entropy_}^* = - \sum_{i=1}^n \frac{w_i p_i}{\sum_{j=1}^n w_j p_j} \log \left(\frac{w_i p_i}{\sum_{j=1}^n w_j p_j} \right)$ |
| ▪ $\text{wtd_mean_}^* = \nu = \sum_{i=1}^n w_i t_i$ | ▪ $\text{range_}^* = \max_i(t_i) - \min_i(t_i)$ |
| ▪ $\text{gmean_}^* = (\prod_{i=1}^n t_i)^{1/n}$ | ▪ $\text{wtd_range_}^* = \max_i(w_i t_i) - \min_i(w_i t_i)$ |
| ▪ $\text{wtd_gmean_}^* = (\prod_{i=1}^n t_i^{w_i})^{1/n}$ | ▪ $\text{std_}^* = \sqrt{\frac{1}{n} \sum_{i=1}^n (t_i - \mu)^2}$ |
| ▪ $\text{entropy_}^* = - \sum_{i=1}^n p_i \log(p_i)$ | ▪ $\text{wtd_std_}^* = \sqrt{\sum_{i=1}^n w_i (t_i - \nu)^2}$ |

donde n es el número de átomos que conforman el compuesto, t_i es el valor de la propiedad para el elemento i , w_i corresponde con la estequiometría normalizada del elemento i , y $p_i = \frac{t_i}{\sum_{j=1}^n t_j}$.

El resultado son 10 familias de descriptores estadísticos, cada una de las cuales está formada por 8 características (una por cada propiedad elemental). Se añade además una nueva propiedad:

- **Número de elementos (*number_of_elements*):** número de elementos químicos distintos presentes en la fórmula del compuesto.

Antes de proceder a la construcción de los modelos predictivos, es fundamental realizar un análisis exploratorio de los datos con el objetivo de comprender con mayor profundidad la estructura del conjunto de datos, evaluar la distribución de las variables y detectar patrones relevantes. En el apéndice A se detalla el estudio llevado a cabo.

4.3. Modelado

Para el modelado se ha decidido inicialmente utilizar *Random Forest* y *XGBoost*, ambos basados en el principio de ensamblado de árboles de decisión. No obstante, se incluye en el estudio un árbol de decisión simple con el objetivo de cuantificar la mejora de rendimiento que proporcionan las técnicas de ensamble.

Tanto el árbol de decisión simple como los modelos base que componen el *Random Forest* emplean el error cuadrático medio (MSE) como criterio de partición durante el entrenamiento. De forma análoga, *XGBoost* optimiza originalmente MSE en su proceso de descenso del gradiente de segundo orden. Todo el código implementado para realizar el estudio puede encontrarse en el repositorio de *GitHub* [13].

4.3.1. Métricas de evaluación

Antes de sumergirnos en el entrenamiento de los modelos, es importante establecer cuáles serán las métricas de evaluación utilizadas para cuantificar su desempeño. Tradicionalmente en problemas de regresión, se han utilizado la raíz cuadrada del error cuadrático medio (*RMSE*) y el coeficiente de determinación (R^2):

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{n}} \quad R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

El *RMSE* proporciona información de la desviación promedio entre los valores predichos por el modelo y los observados experimentalmente. Por su parte, R^2 indica la proporción de la variabilidad de resultados que es explicada por el modelo, por ello, sirve como una medida de la calidad del ajuste.

Se contempla también el uso de una métrica más. El error porcentual absoluto medio (*MAPE*) nos aportará también una medida de la precisión relativa de las estimaciones:

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{\hat{y}_i - y_i}{y_i} \right|$$

El $MAPE$ se calcula como el promedio del error porcentual absoluto con respecto al valor experimental real. Es importante tener en cuenta que nuestra variable objetivo abarca varios órdenes de magnitud, con valores que van desde 0,00021 K , hasta 185 K . Al evaluar nuestros resultados mediante el $MAPE$, los errores absolutos pequeños, dado el rango de valores que puede tomar la temperatura crítica, provocarán que, para las muestras en la región de temperaturas bajas, esta métrica crezca de forma descontrolada.

Por esta razón, se decide utilizar el error porcentual absoluto medio simétrico $sMAPE$, que corrige este sesgo al normalizar el error con respecto a la media entre el valor real y el estimado, proporcionándonos una medida más equilibrada.

$$sMAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{\hat{y}_i - y_i}{(\hat{y}_i + y_i)/2} \right|$$

4.3.2. Evaluación previa

Antes de comenzar el entrenamiento de los modelos, se realiza la partición de los datos en dos subconjuntos: uno destinado a entrenamiento y otro a la validación, que permitirán evaluar correctamente el rendimiento de los modelos.

Con el objetivo de seleccionar los algoritmos con mejor desempeño antes del ajuste final, se procede con una evaluación preliminar mediante validación cruzada sobre el conjunto de entrenamiento. En la Tabla 4.1 se pueden ver los resultados obtenidos.

Tabla 4.1: Métricas de evaluación obtenidas para la validación cruzada sobre el conjunto de entrenamiento.

	$RMSE$	R^2	$sMAPE$
Random Forest	9.718	0.920	26.373
XGBoost	9.787	0.919	32.424
Decision Tree	12.403	0.869	29.186

Como podemos ver reflejado en el $RMSE$, las predicciones del árbol de decisión simple son, en términos absolutos, de peor calidad que las obtenidas por los métodos de ensamble. No obstante, resulta llamativo el resultado obtenido por este modelo en la métrica $sMAPE$, pues logra superar al *eXtreme Gradient Boosting*. La explicación reside en que el $sMAPE$ es menos sensible que el $RMSE$ a valores extremos y, puesto que los árboles de decisión simples tienden a generar predicciones más conservadoras y centradas, reducen los errores relativos porcentuales. A pesar de ello, este modelo no consigue explicar con la misma eficacia que los algoritmos de ensamble la variabilidad de la temperatura crítica, tal como refleja su R^2

Los resultados obtenidos por los modelos basados en técnicas de ensamble son, en cambio, bastante prometedores, considerando el rango de valores que puede adoptar la variable objetivo.

Ambos alcanzan un coeficiente de determinación R^2 superior a 0,9 y un $RMSE$ inferior a 9,8. Por este motivo, se decide prescindir del árbol de decisión simple para las fases posteriores del estudio, centrando el análisis en los dos modelos con mayor potencial.

4.4. Resultados

Se procede ahora al entrenamiento de los modelos basados en métodos de ensamble sobre la totalidad del conjunto de entrenamiento, con el fin de evaluar su rendimiento de forma más representativa. En esta fase, la evaluación se da únicamente sobre el conjunto de validación, por lo que los resultados deben mostrar una ligera mejora respecto a los obtenidos durante la evaluación previa. Esto es debido a que, al emplearse un mayor volumen de datos durante el entrenamiento, mejora el ajuste de los modelos a las relaciones subyacentes. En la Tabla 4.2 podemos observar los resultados obtenidos.

Tabla 4.2: Métricas de evaluación obtenidas para el entrenamiento general y calculadas sobre el conjunto de validación.

	$RMSE$	R^2	$sMAPE$
Random Forest	9.042	0.929	25.187
XGBoost	8.973	0.925	32.975

Como podemos apreciar, ambos modelos presentan una mejora general respecto a la evaluación previa. Comprobamos una disminución del $RMSE$ y un incremento en R^2 en ambos algoritmos. *Random Forest* logra un desempeño ligeramente mayor en términos de $sMAPE$. En conjunto, los resultados confirman que los métodos de ensamble ofrecen una capacidad predictiva sólida para este problema de regresión.

Sabemos que, escogiendo los hiperparámetros adecuados, es posible lograr una mejora en la capacidad predictiva de los modelos. Los algoritmos utilizados proporcionan herramientas que, por procesos heurísticos y a través de validación cruzada, son capaces de encontrar los hiperparámetros que optimizan el rendimiento del modelo. En la Tabla 4.3 se pueden consultar los resultados obtenidos tras esta optimización.

Tabla 4.3: Métricas de evaluación obtenidas tras el tuneo de hiperparámetros y calculadas sobre el conjunto de validación.

	$RMSE$	R^2	$sMAPE$
Random Forest	8.979	0.930	25.740
XGBoost	8.668	0.935	27.606

Tras el ajuste, ambos modelos mantienen un rendimiento estable, presentando una mejora ligera con respecto al entrenamiento inicial. El algoritmo *XGBoost* alcanza los mejores resultados en términos de R^2 y $RMSE$, consolidándose como el de mayor capacidad explicativa global. Por su parte, el *Random Forest* presenta un $sMAPE$ inferior, es decir, en términos relativos, sus predicciones son más estables.

En la Figura 4.1 se muestran los gráficos comparativos entre los valores reales y predichos por cada modelo. De esta forma se puede analizar visualmente la dispersión de las predicciones en cada rango de temperaturas críticas.

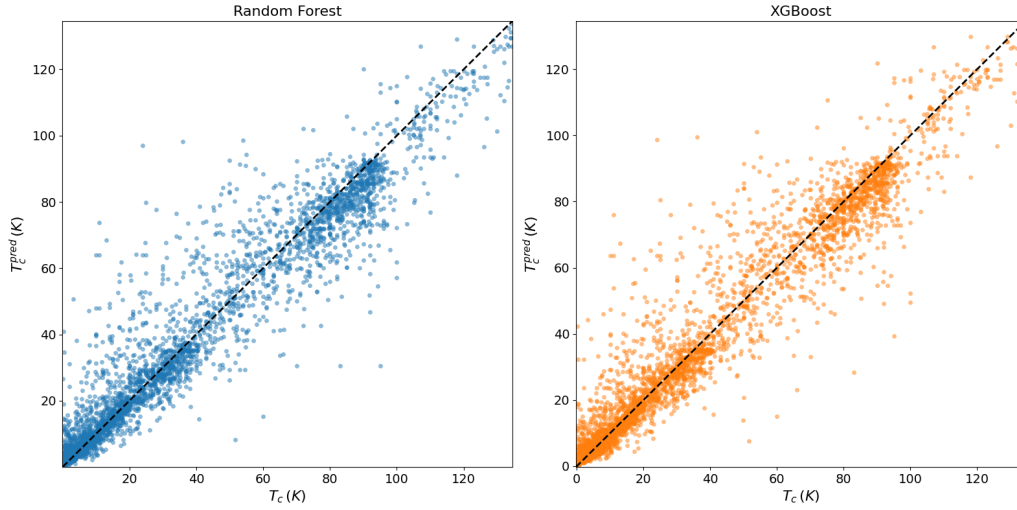


Figura 4.1: Comparación entre valores reales y predichos. *Random Forest* a la izquierda y *XGBoost* a la derecha.

Analizando brevemente la Figura 4.1, comprobamos cómo los puntos se concentran en torno a la diagonal, que precisamente correspondería con las predicciones perfectas. No obstante, podemos observar predicciones realmente alejadas de sus respectivos valores reales. La dispersión en la región de la diagonal, así como los valores mal predichos, explican que los resultados obtenidos para el *sMAPE* superen el 25 %.

Los modelos de ensamble basados en árboles son capaces de obtener una estimación de la importancia de cada variable en el proceso de predicción. En la Figura 4.2 se detallan las importancias calculadas por nuestros modelos. Esta medida se obtiene a partir del número de divisiones en las que participa un determinado descriptor en la totalidad de árboles que componen en ensamble. Dependiendo del método de ensamble, es una medida más o menos certera, esto se debe a que el diseño del algoritmo puede afectar a la proporcionalidad entre la frecuencia de uso de una variable y la relevancia real en la relación subyacente.

De la Figura 4.2 podemos extraer ciertas conclusiones: la conductividad térmica de los elementos que componen el material superconductor adquiere una gran relevancia. Particularmente, el rango entre los valores máximo y mínimo de esta propiedad se encuentra entre los descriptores con mayor peso en ambos modelos. Conviene ahora rescatar los resultados del análisis exploratorio de los datos A, en el que ya se había intuido la relevancia de este descriptor. El radio atómico de los elementos del compuesto y, en particular, su rango de variabilidad, también presenta una contribución significativa en la predicción de la temperatura crítica. Por último, cabe destacar las apariciones de la media ponderada de la valencia en *Random Forest*, y de la desviación estándar ponderada de la afinidad electrónica en *XGBoost*. Otras propiedades como la masa atómica o la primera energía de ionización (*fi*) tienen también una relevancia notoria.

Para el caso de *XGBoost* se puede comprobar cómo el rango de variabilidad de la conductividad

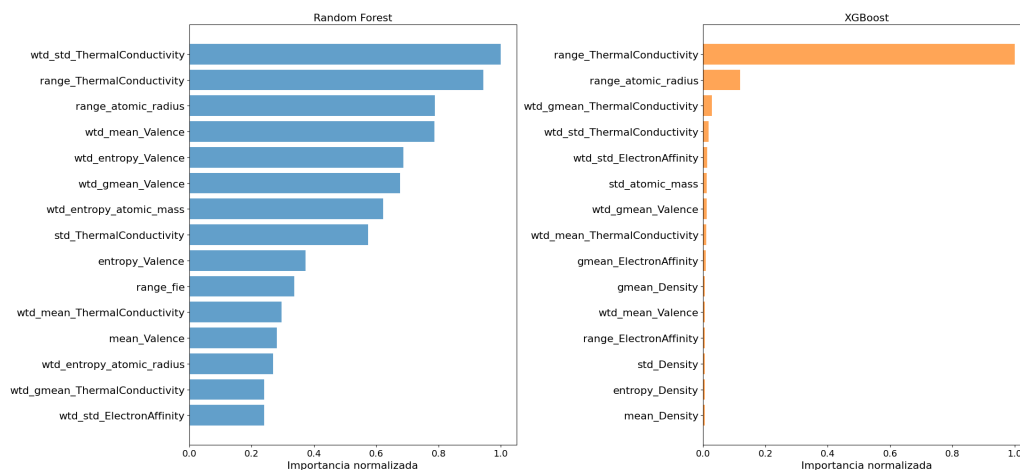


Figura 4.2: Variables más relevantes para cada modelo. *Random Forest* a la izquierda y *XGBoost* a la derecha.

térmica concentra una proporción muy elevada de la importancia total. Este comportamiento puede atribuirse al mecanismo de aprendizaje de este método de ensamble, que focaliza la creación de nuevos árboles en las observaciones más difíciles de predecir. El resultado es que los descriptores más útiles para corregir los errores previos aparecen con mayor frecuencia, lo que finalmente se ve reflejado en la importancia relativa.

4.5. Conclusiones

Los resultados obtenidos demuestran la capacidad de estos modelos para capturar las tendencias globales del fenómeno estudiado, prueba de ello es un *RMSE* inferior a 9 y un R^2 por encima de 0,9 en ambos modelos. Sin embargo, a nivel de predicción individual, su precisión resulta más limitada, tal como reflejan las métricas relativas como el *sMAPE*, la cual no ha podido reducirse por debajo del 25 %.

Por otro lado, la interpretabilidad y la posibilidad de analizar la importancia de las variables constituyen una ventaja significativa en contextos de investigación científica, donde comprender las relaciones subyacentes es tan relevante como predecir con exactitud.

En conjunto, este proyecto nos permite comprender el funcionamiento de diversas herramientas de aprendizaje automático y su aplicación a un caso científico real. En este estudio se han expuesto las bases del *Machine Learning*, los fundamentos matemáticos que sustentan los árboles de decisión y las principales técnicas de ensamble: *stacking*, *bagging* y *boosting*. Para finalizar, hemos llevado dos de los algoritmos estudiados al terreno práctico.

En un panorama dominado por el auge de la inteligencia artificial moderna y los modelos de *Deep Learning*, este trabajo pone en valor el potencial y la vigencia de los métodos basados en árboles de decisión. Aunque no siempre alcanzan el rendimiento de las redes neuronales en grandes volúmenes de datos, su equilibrio entre precisión, robustez y explicabilidad los convierten en herramientas especialmente adecuadas para problemas científicos con conjuntos de datos acotados y alta necesidad de interpretabilidad.

Bibliografía

- [1] G. James, D. Witten, T. Hastie, R. Tibshirani y J. Taylor, *An Introduction to Statistical Learning: with Applications in Python*, 2023rd. New York, NY: Springer, 2023, ISBN: 978-3031387463. dirección: <https://www.statlearning.com/>
- [2] L. Breiman, J. H. Friedman, R. A. Olshen y C. J. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth International Group, 1984.
- [3] F. Pedregosa et al., «Scikit-learn: Machine Learning in Python,» *Journal of Machine Learning Research*, vol. 12, págs. 2825-2830, 2011.
- [4] T. Hastie, R. Tibshirani y J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2.^a ed. New York, NY: Springer, 2009, ISBN: 978-0387848570. dirección: <https://web.stanford.edu/~hastie/ElemStatLearn/>
- [5] R. E. Schapire, «The Strength of Weak Learnability,» *Machine Learning*, vol. 5, n.º 2, págs. 197-227, 1990. DOI: [10.1007/BF00116037](https://doi.org/10.1007/BF00116037)
- [6] D. H. Wolpert, «Stacked Generalization,» *Neural Networks*, vol. 5, n.º 2, págs. 241-259, 1992. DOI: [10.1016/S0893-6080\(05\)80023-1](https://doi.org/10.1016/S0893-6080(05)80023-1)
- [7] L. Breiman, «Bagging Predictors,» *Machine Learning*, vol. 24, n.º 2, págs. 123-140, 1996. DOI: [10.1007/BF00058655](https://doi.org/10.1007/BF00058655)
- [8] L. Breiman, «Random Forests,» *Machine Learning*, vol. 45, n.º 1, págs. 5-32, 2001. DOI: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324)
- [9] Y. Freund y R. E. Schapire, «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting,» *Journal of Computer and System Sciences*, vol. 55, n.º 1, págs. 119-139, 1997. DOI: [10.1006/jcss.1997.1504](https://doi.org/10.1006/jcss.1997.1504)
- [10] J. H. Friedman, «Greedy Function Approximation: A Gradient Boosting Machine,» *The Annals of Statistics*, vol. 29, n.º 5, págs. 1189-1232, 2001. DOI: [10.1214/aos/1013203451](https://doi.org/10.1214/aos/1013203451) dirección: <https://projecteuclid.org/euclid.aos/1013203451>
- [11] T. Chen y C. Guestrin, «XGBoost: A Scalable Tree Boosting System,» en *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, New York, NY, USA: Association for Computing Machinery, 2016, págs. 785-794. DOI: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785)

- [12] K. Hamidieh, «A data-driven statistical model for predicting the critical temperature of a superconductor,» *Computational Materials Science*, vol. 154, págs. 346-354, 2018. DOI: [10.1016/j.commatsci.2018.07.052](https://doi.org/10.1016/j.commatsci.2018.07.052) dirección: <https://doi.org/10.1016/j.commatsci.2018.07.052>
- [13] M. Barba. «BarbaMarcos_TFG.» Repositorio en GitHub, visitado 10 de nov. de 2025. dirección: https://github.com/marcosbarba/BarbaMarcos_TFG

Apéndice A

Análisis exploratorio de los datos

No se encontró ninguna variable con valor nulo, por lo que no es necesario aplicar imputación ni eliminar ningún registro. Todas las variables son numéricas y, a excepción del número de elementos y el rango de valencias, son todas continuas. No hay que utilizar técnicas de codificación, ya que no hay variables categóricas. El número de elementos y el rango de valencias son variables discretas y, además, presentan un número limitado de valores posibles.

Análisis univariante

Se realiza un análisis individualizado de las variables con el objetivo de estudiar su comportamiento de forma independiente. Para esta primera exploración debemos seleccionar una serie de características representativas del conjunto, pues el análisis de las 81 variables resultaría excesivamente extenso. La familia de descriptores *mean_** es una de las más descriptivas y fáciles de interpretar, por lo que se seleccionan estas 8 variables, además de *number_of_elements*. El análisis univariante se realiza también sobre la variable objetivo *critical_temp*.

Este primer examen nos permite observar el rango, la dispersión y la forma de la distribución de cada propiedad. Así podemos detectar sesgos, diferencias de escala y valores atípicos, aspectos a tener en cuenta en las fases de preprocesamiento y modelado.

En primer lugar, analizamos la distribución de la variable objetivo.

Como se observa en la Figura A.1, la distribución de la temperatura crítica está sesgada a la derecha con la mayoría de los superconductores concentrados en el rango de temperaturas críticas entre 0 K y 50 K. No obstante, la distribución se alarga hasta valores por encima de los 130 K, aunque se observa un claro descenso por encima de los 100 K. La distribución refleja la realidad experimental: los superconductores de alta temperatura crítica son excepcionales.

El resultado es un dataset desbalanceado, no por errores de muestreo, sino por la realidad física de este fenómeno. Este desequilibrio deberá tenerse en cuenta durante el modelado, ya que los algoritmos tenderán a sesgarse y ajustar mejor los compuestos con temperaturas críticas bajas.

Advertimos la presencia de un outlier con $T_c = 185\text{ K}$ que corresponde con el compuesto H_2S .

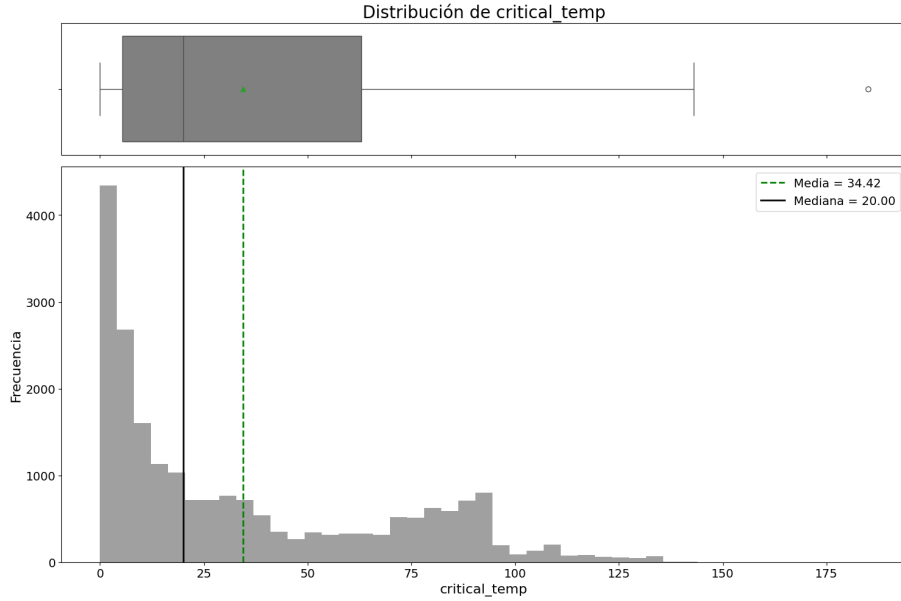


Figura A.1: Distribución de la variable objetivo T_c .

No obstante, al tratarse de un outlier físico, no se elimina del conjunto de datos. Esta decisión se basa en que los modelos que se van a utilizar, basados en árboles de decisión, son robustos frente a la presencia de outliers.

En la Figura A.2 podemos visualizar distribuciones con notables sesgos positivos, como es el caso de *mean_Density*, *mean_FusionHeat* y *mean_Valence*, hasta distribuciones más simétricas como *mean_atomic_mass*, *mean_fie* o *mean_ElectronAffinity*. En cambio, *mean_ThermalConductivity* y *mean_atomic_radius* muestran un leve sesgo negativo.

En general, las variables presentan una dispersión considerable, con rangos amplios y la presencia de valores atípicos en la mayoría de los casos, especialmente en regiones superiores. Es importante notar que estos valores extremos no provienen de errores experimentales, sino de outliers físicos reales. Por ello se toma la decisión de mantenerlos, pues eliminarlos implicaría perder información física valiosa de comportamientos excepcionales, pero posibles.

Análisis bivariente

En este punto nos centramos en las correlaciones entre descriptores y en particular, su relación con la variable objetivo *critical_temp*.

Este estudio permite observar cómo varía la temperatura crítica en función de las distintas propiedades atómicas y estructurales de los materiales y detectar tendencias y patrones físicos relevantes.

En primer lugar, se realiza una exploración visual de las relaciones de las variables explicativas de la familia *mean_** y la variable objetivo. Este análisis lo hacemos a través de mapas de densidad, a través de los cuales podemos visualizar para qué valores se concentra una mayor cantidad de superconductores.

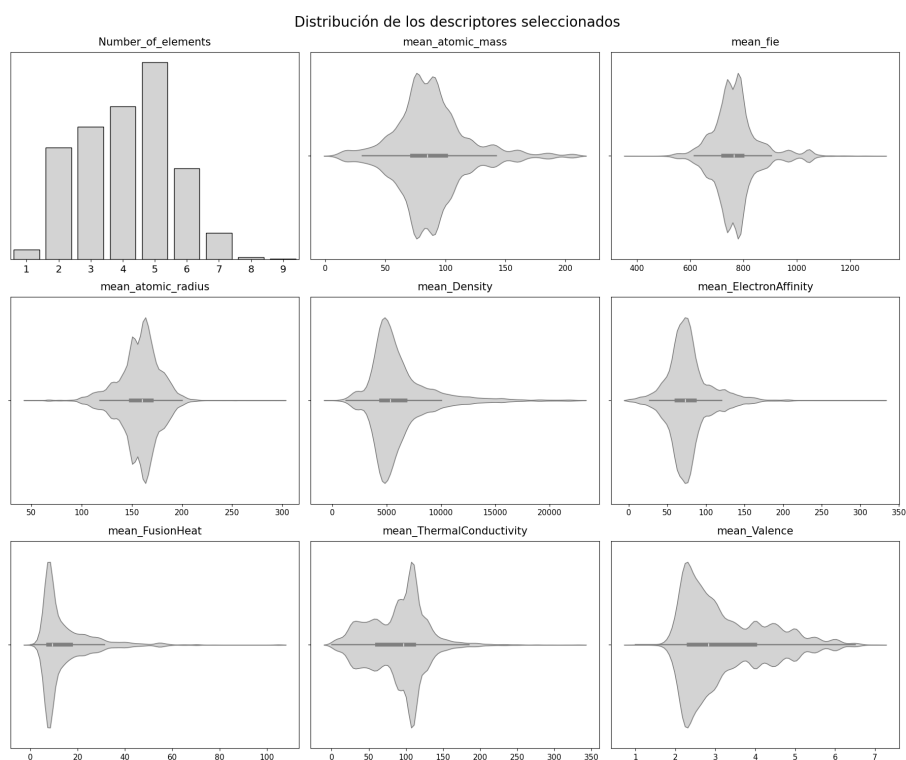


Figura A.2: Distribuciones de los descriptores seleccionados para el análisis univariante.

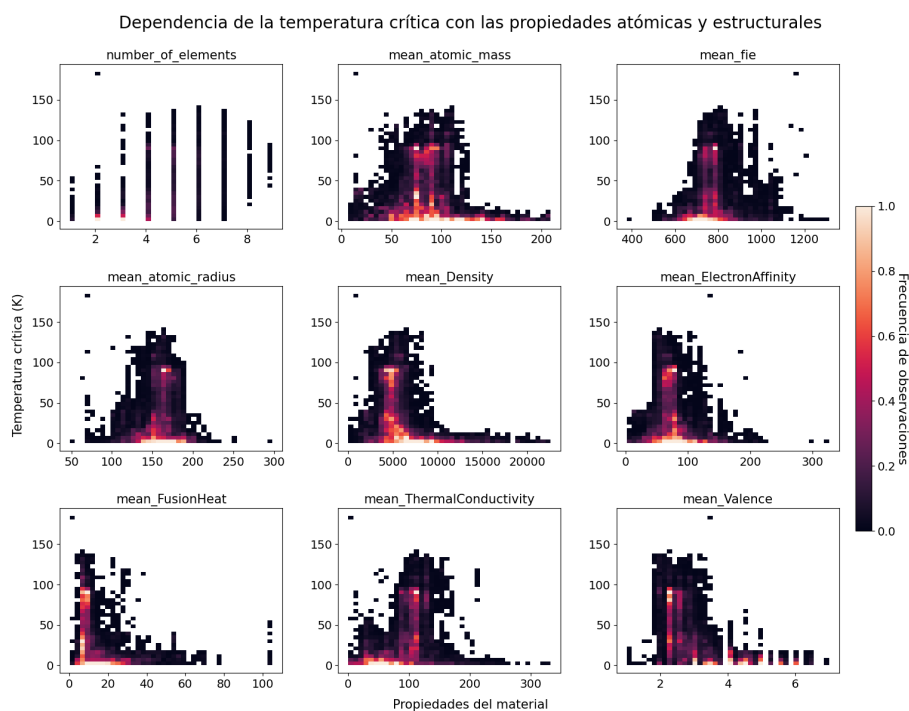


Figura A.3: Mapas de densidad para las relaciones entre los descriptores seleccionados y la variable objetivo.

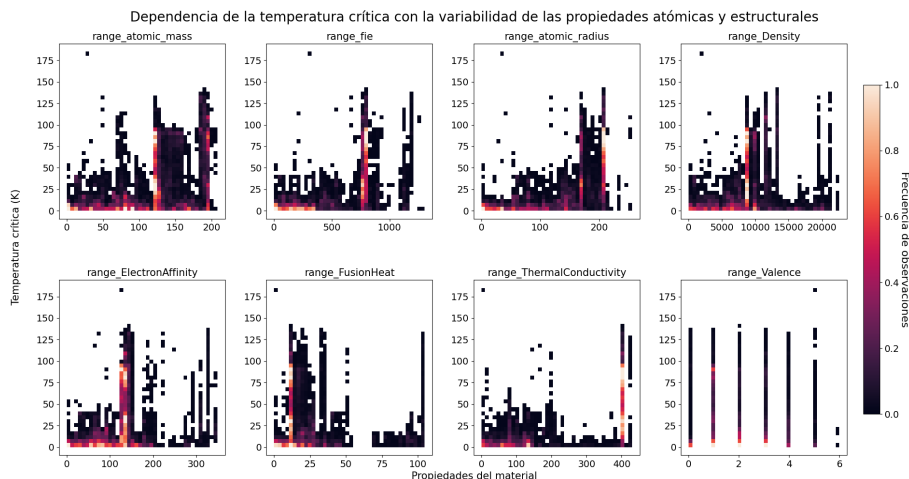


Figura A.4: Mapas de densidad para las relaciones entre los descriptores de la familia *range_** y la variable objetivo.

En la Figura A.3 se identifican zonas de concentración local de superconductores de alta temperatura. Esto ocurre generalmente en las regiones de mayor densidad de muestras (tanto de baja como de alta temperatura). Esto es, las temperaturas críticas altas se dan en subconjuntos específicos dentro de los rangos más comunes.

Para esta parte del análisis, en la que se pretende profundizar en qué propiedades resultan más determinantes en la variación de la temperatura crítica de los materiales superconductores, se ha decidido investigar la correlación de la variable objetivo con la familia *range_**.

En la Figura A.4 se observan tendencias más marcadas que las vistas anteriormente para la familia *mean_**. En particular, se aprecian intervalos concretos de estos descriptores que muestran una concentración elevada de superconductores de alta temperatura. Este comportamiento se refleja en *range_fie*, *range_ElectronAffinity* y, de forma más notoria, en *range_ThermalConductivity* que presenta una región relativamente limpia en la que se agrupan la mayoría de compuestos con temperaturas críticas elevadas, salvo algunas muestras aisladas fuera de ella.

Si bien este análisis permite identificar ciertos patrones en la relación entre las propiedades atómicas y estructurales y la temperatura crítica, no se identifican relaciones funcionales evidentes. Esto sugiere que el fenómeno superconductor depende de interacciones no lineales y multivariantes. Este comportamiento justifica la necesidad de emplear modelos de aprendizaje automático, capaces de capturar interacciones complejas entre múltiples variables.

Continuando nuestro análisis, adoptamos una perspectiva más estadística con el objetivo de explorar no solo la correlación existente entre cada una de las propiedades con la temperatura crítica, sino también las posibles dependencias entre los propios predictores. Para ello calculamos la matriz de correlación de la familia *mean_**.

En la Figura A.5 podemos advertir correlaciones entre las propiedades, todas ellas esperables desde el punto de vista físico-químico. Un ejemplo claro es la correlación positiva de *mean_atomic_mass* con *mean_atomic_radius* y *mean_Density* o la correlación negativa entre *mean_atomic_radius* y *mean_fie* (a mayor tamaño del átomo, menor atracción sobre los elec-

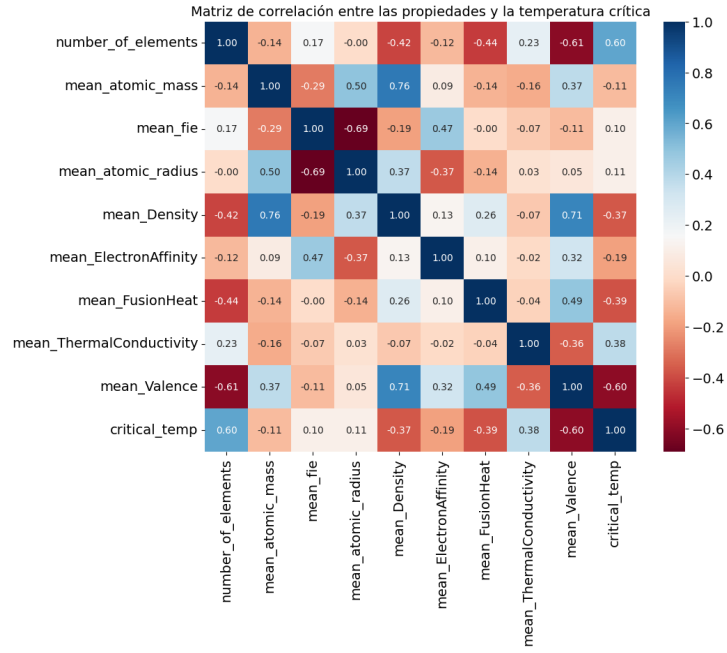


Figura A.5: Matriz de correlación correspondiente a los descriptores seleccionados y la variable objetivo.

trones de valencia y menor energía de ionización).

Respecto a la variable objetivo, se observa correlación positiva notable con *number_of_elements* y algo más moderada con *mean_ThermalConductivity*. Con *mean_Valence* se tiene una correlación negativa relativamente fuerte, y en menor medida con *mean_FusionHeat* y *mean_Density*. El resto de variables presentan correlaciones débiles con la temperatura crítica.

Dadas las relaciones observadas anteriormente para la familia *range_**, consideramos también la obtención de su respectiva matriz de correlación, incluyendo también la variable objetivo.

Como podíamos esperar basándonos en la relación observada entre *range_ThermalConductivity* y la variable objetivo, obtenemos una correlación positiva considerable para este par. Son también destacables las correlaciones positivas de *range_fie* y [*range_atomic_radius*] con *critical_temp*.

En la Figura A.7 se incluye la matriz de correlación completa entre todas las variables del conjunto de datos. Como consecuencia de la gran cantidad de descriptores, se pierde detalle a nivel individual, pero podemos identificar patrones globales de dependencia. Vemos reforzadas las correlaciones ya observadas en el subconjunto analizado. No obstante, aparecen nuevas correlaciones interesantes, especialmente al considerar las familias *range_** y *std_**, lo que nos indica que, para ciertos materiales, la variabilidad en las propiedades atómicas es un factor relevante para determinar la temperatura crítica.

Este análisis también evidencia la alta multicolinealidad entre ciertas familias, como es el caso de *mean_**, *wtd_mean_**, *gmean_** y *wtd_gmean_**. No obstante, los modelos basados en árboles que se van a utilizar son capaces de gestionar internamente la redundancia y seleccionar las características más relevantes durante el entrenamiento.

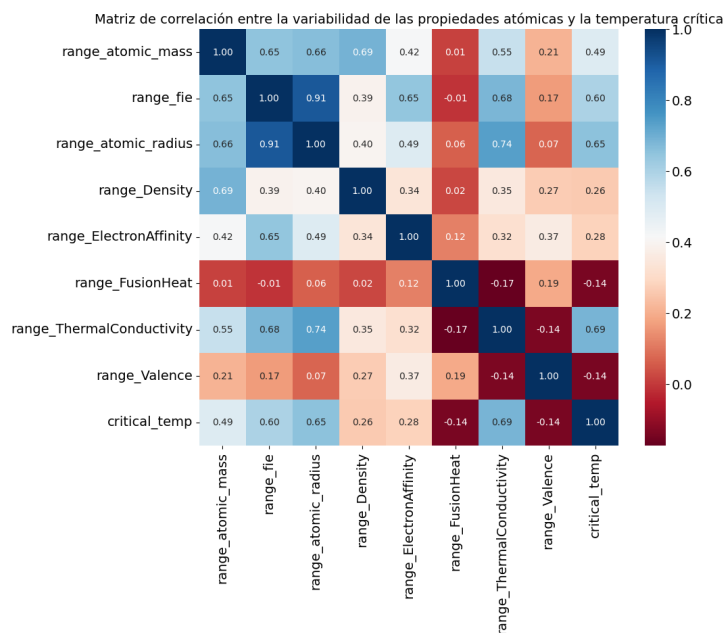


Figura A.6: Matriz de correlación correspondiente a los descriptores de la familia *range_** y la variable objetivo.

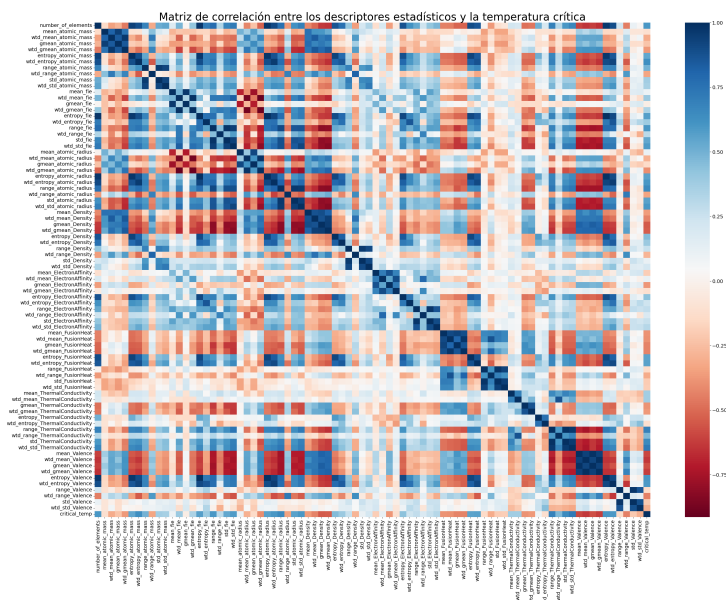


Figura A.7: Matriz de correlación completa. Se incluyen los 81 descriptores y la variable objetivo.